

Machine Learning en finanzas cuantitativas

Clase 6: El pipeline riguroso

Guido Schifani

FCEN · UBA



▶ Teoría de portafolio: de un activo a la cartera

- Markowitz: optimizamos media-varianza con la frontera eficiente. Insumos: μ (retornos esperados) y Σ (covarianzas)
- CAPM como equilibrio: si todos son Markowitz y el mercado limpia, β explica el retorno esperado
- Shrinkage (Ledoit-Wolf), Black-Litterman, risk parity: cómo trabajar el ruido en $\hat{\Sigma}$

▶ El problema que dejamos abierto: ¿de dónde vienen los μ ?

- Markowitz toma μ como input. Estimarlos con la media histórica tiene un error estándar de $\sigma/\sqrt{T}$ — enorme en finanzas
- Black-Litterman reemplaza $\hat{\mu}$ por retornos de equilibrio + views. Pero ¿se pueden aprender views del mercado?

▶ La promesa del ML: aprender retornos esperados condicionales al estado del mercado

- Si existe señal predecible (aunque sea débil), un modelo puede captarla mejor que la media histórica
- El problema: el pipeline estándar de ML, aplicado a finanzas tal cual, produce modelos que parecen funcionar y pierden plata

§1

Por qué el ML de manual falla en finanzas

Los siete problemas estructurales · El adversario adaptativo · Calibrar expectativas

▶ Clasificación

- ¿Este activo supera al benchmark en $T+1$? ¿Cambia el régimen de volatilidad?
- ¿Qué lado toma el próximo tick del order book? (T_0 , el problema del market maker)

▶ Regresión

- IV futura de un strike (para tradeear vega con calendars); la smile completa como curva
- Retorno esperado de un activo condicional a otros, insumo de scalping (operar posiciones muy cortas, segundos a minutos, capturando pequeñas diferencias de precio) o market making

▶ Clustering

- Segmentar el universo ANTES de entrenar: un modelo por cluster en vez de uno para las 500 del S&P

▶ Redes y RL

- RNN/LSTM en contextos cortos, ¿el precio es Markoviano, o hay memoria real que explotar?
- RL para minimizar impacto de ejecución (órdenes iceberg), retoma en microestructura, clase 7

- ▶ **Momentum y reversión a la media: las dos ideas que generan la mayoría de las señales**
 - Momentum (trend-following): el precio subió N días → señal de 'seguir'. Son señales tardías: confirman después del movimiento
 - Contrarian (reversión a la media): la lógica inversa: 'subió demasiado rápido → más probable que vuelva al promedio'
 - Ejemplo: RSI (Relative Strength Index). Compara el promedio de días que subió vs. días que bajó en una ventana de N días. $RSI > 70$ = sobrecomprado (posible caída); $RSI < 30$ = sobrevendido (posible rebote). Siempre entre 0 y 100
- ▶ **Flujo e información institucional**
 - Volumen, put/call ratio, open interest: cuánta actividad hay y qué apuestan los traders de opciones
 - 13F: los fondos de inversión con más de \$100M en activos deben reportar sus posiciones a la SEC cada trimestre. El 13F es ese reporte público. Es una señal trimestral de hacia dónde apuestan los institucionales
- ▶ **Fundamentales y contexto macro**
 - P/E, P/B, beta contra el S&P, correlaciones con FX y commodities. Contextualizan la señal de precio con la realidad económica del activo
- ▶ Regla fundamental: más features NO es mejor. Un modelo con 5 features limpias suele ganarle a uno con 200 ruidosas

El pipeline que ya saben y por qué acá produce pérdidas

▶ Lo que un data scientist hace por reflejo

- Datos → features → modelo → cross-validation → accuracy alto → producción
- Ese pipeline, aplicado a finanzas TAL CUAL, produce modelos que parecen buenísimos y pierden plata

▶ Los siete problemas estructurales

- 1. Datos NO son IID: sklearn asume independencia y estacionariedad entre muestras. Los retornos tienen autocorrelación, clustering de vol, y el mercado de 2010 ≠ el de 2020
- 2. Señal/ruido ínfimo: en 252 días hábiles, acertar el 51% implica 127 aciertos vs 125 fallos: ¿suerte o habilidad? El 55% (275 días de más en 5 años) ya es extraordinario.
- 3. No estacionariedad: el modelo entrenado sobre el mercado de 2015 puede ser inútil en 2020 (COVID), el pasado caduca endógenamente
- 4. Leakage: el futuro se filtra al entrenamiento por vías invisibles
- 5. Overfitting: con señal/ruido ínfimo, cualquier modelo flexible memoriza el ruido
- 6. Multiple testing: si probaste 100 variantes y elegiste la mejor, está sesgada
- 7. El backtest es irreplicable: hay una sola historia, no se puede 'probar con nuevos datos del pasado'

En qué se diferencia de toda otra ciencia

▶ En física, el sistema no lee tus papers

- El electrón se comporta igual después de que publicás su masa
- El mercado NO: si tu patrón funciona y se conoce, otros lo operan, y el patrón desaparece
- McLean-Pontiff (clase 4): los factores publicados pierden ~50% del premio POST-publicación

▶ Consecuencias para el que modela

- La no estacionariedad no es ruido de fondo: es ENDÓGENA, el mercado se re-modela contra vos
- Tu competencia lee los mismos libros, baja los mismos datos, corre los mismos modelos
- La ventaja sostenible viene de: datos que otros no tienen, rigor que otros no aplican, o velocidad
- Hoy vemos el RIGOR

▶ Calibrar expectativas, los números honestos

- Un Sharpe out-of-sample de 0.5 SOSTENIDO ya es muy bueno; 1.0 es excelente; 2+ es sospechoso
- Si un backtest te muestra Sharpe 3, la pregunta no es 'cuánto invierto' sino 'dónde está el bug'

► El flujo de trabajo, de los datos crudos a la estrategia auditada

- §2 LOS DATOS: ¿cómo muestreo el mercado? → dollar bars (datos no IID)
- §3 LAS ETIQUETAS: ¿qué le pregunto al modelo? → triple barrera + meta-labeling
- §4 LOS PESOS: ¿cuánta información independiente tengo? → unicidad + diferenciación fraccionaria
- §5 LA VALIDACIÓN: ¿cómo mido sin mentirme? → purged K-fold + embargo (leakage temporal)
- §6 EL MODELO Y SUS FEATURES: bagging vs boosting; ¿qué aprendió? → MDI / MDA (interpretabilidad)
- §7 EL VEREDICTO: ¿es señal o suerte? → Deflated Sharpe + PBO (multiple testing)

► El orden importa: cada etapa asume que la anterior se hizo bien

- El error típico: sofisticar el modelo (etapa 6) con etiquetas malas y validación incorrecta
- Lopez de Prado invierte la prioridad: los modelos son lo de menos (un random forest alcanza); el rigor está en los datos, las etiquetas y la validación



§2

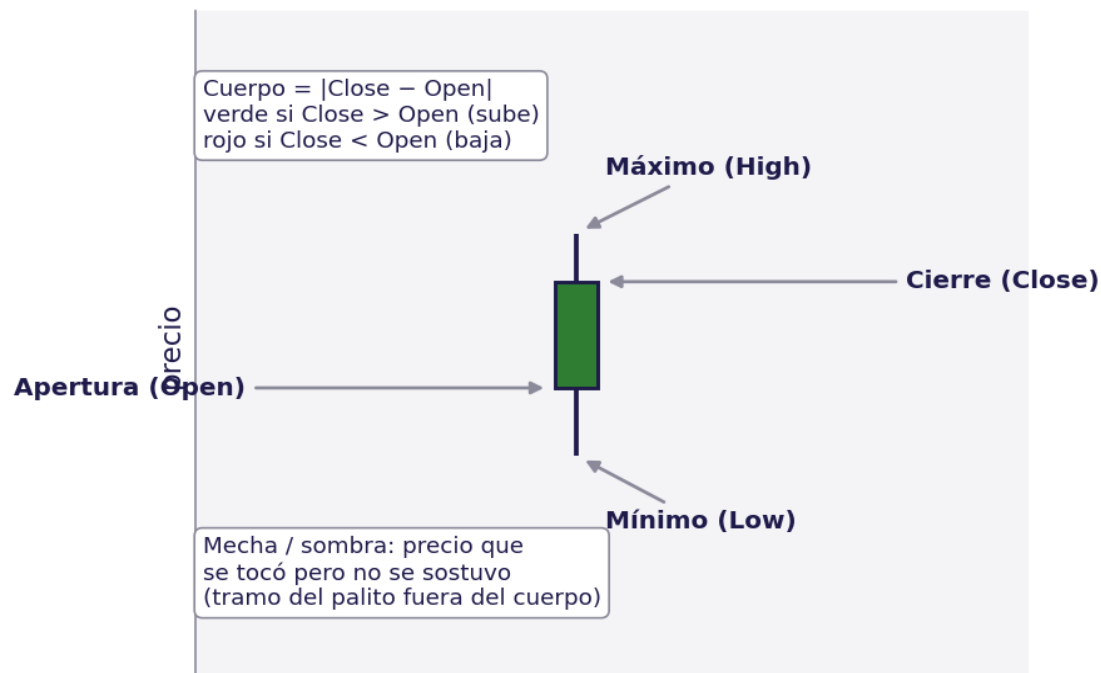
Los datos: muestrear por actividad, no por reloj

El problema de las time bars · Tick, volume y dollar bars

¿Qué es una vela (bar)? El bloque atómico de toda serie de precios

¿Qué es una vela (bar)? El bloque atómico de cualquier serie de precios

**Una vela = O, H, L, C
el resumen de un intervalo de actividad**



**Una secuencia de velas: así se lee un gráfico
(serie sintética, semilla fija)**



Cada vela resume un intervalo de actividad con cuatro números: Open (precio al inicio), High y Low (extremos tocados), Close (precio al final). El cuerpo (rectángulo) es el rango Open–Close y se colorea según el signo del movimiento; la mecha (o sombra) es lo que el precio tocó y no sostuvo. Lo único que cambia entre time, tick, volume y dollar bars es la REGLA que decide cuándo cerrar una vela y abrir la siguiente — el resto de la clase da por sabido este dibujo.

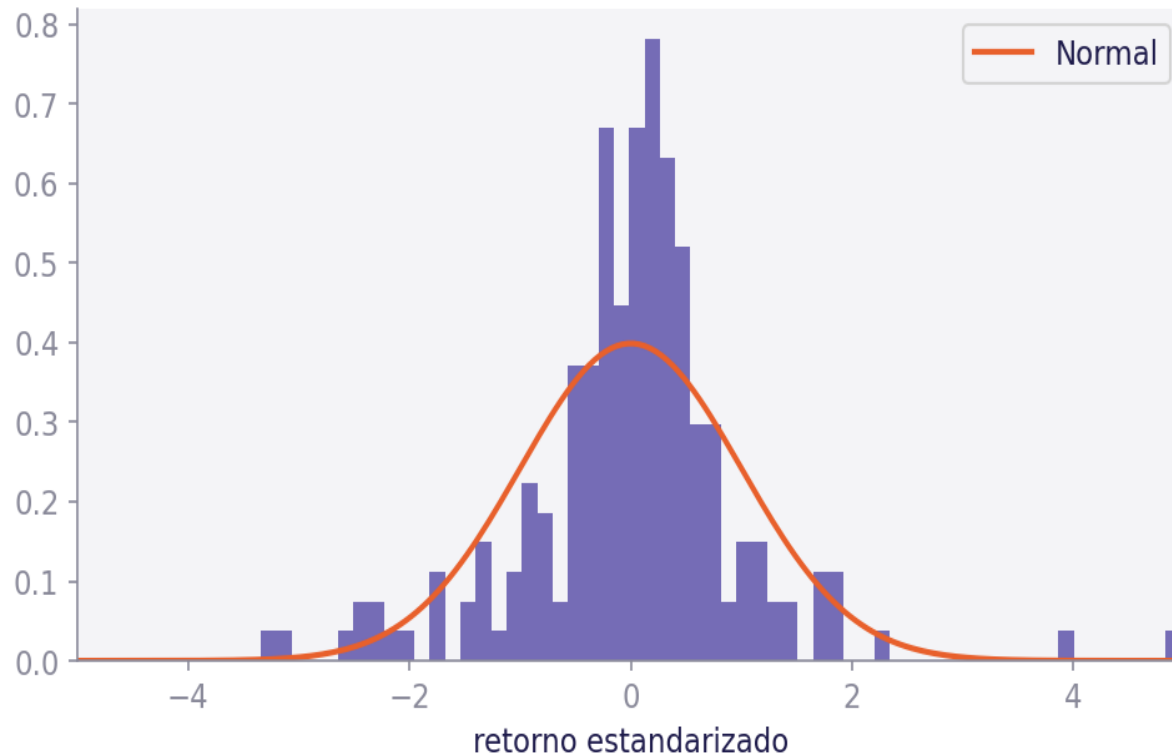
- ▶ **Lo que TODOS usan: velas de 1 día, 1 hora, 5 minutos, cortar el tiempo en pedazos iguales**
- ▶ **El problema: el mercado no genera información a ritmo de reloj**
 - 12:30 de un martes de agosto: no pasa NADA, pero la time bar registra un punto igual
 - 9:30 de un día de Fed, o un flash crash: pasa TODO, y sigue siendo un solo punto
 - Resultado: sobre-muestreás la siesta y sub-muestreás el pánico
- ▶ **La consecuencia estadística (que ya conocen de la clase 1)**
 - Retornos de time bars: colas pesadas, volatilidad agrupada, autocorrelación en r^2
 - Exactamente las propiedades que rompen los supuestos de cualquier modelo de ML
- ▶ **Solución**
 - ¿Y si el reloj corriera al ritmo de la ACTIVIDAD del mercado, no del tiempo físico?
 - En el 'reloj de mercado', los retornos se vuelven mucho más gaussianos e independientes
- ▶ Analogía física: muestrear por EVENTOS (cada colisión) en vez de por segundos

- ▶ **Tick bars: una vela cada N operaciones**
 - El reloj más simple: cuenta transacciones, sin importar su tamaño
 - Defecto: una orden de 10.000 acciones partida en 100 ticks 'infla' la actividad
- ▶ **Volume bars: una vela cada N acciones operadas**
 - Corrige la fragmentación de órdenes, mejor que ticks
 - Defecto: 100 acciones de 2010 $\neq$ 100 acciones de 2026 (el precio cambió 5x)
- ▶ **Dollar bars: una vela cada N DÓLARES operados**
 - Robusto a splits, a la inflación del precio, y comparable entre activos y épocas
- ▶ **La recompensa, medible**
 - Retornos de dollar bars: menos kurtosis, menos autocorrelación en r^2 , más cerca de IID-normal
 - Mejor materia prima para ML: los supuestos del modelo se violan menos

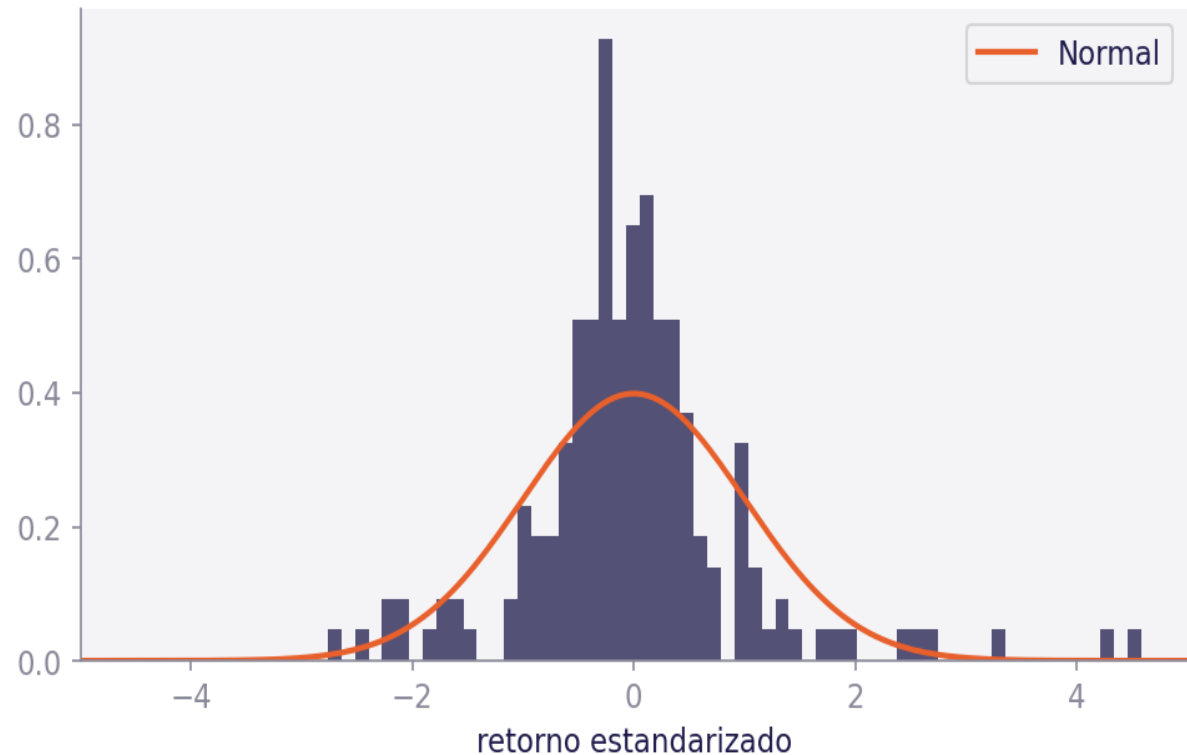
Time bars vs dollar bars: el mismo mercado, dos relojes

Muestrear por actividad, no por reloj — retornos de SPY 5-min (Yahoo, 30 días)

Time bars (1 h de reloj)
exceso de kurtosis = 4.6 (n=194)



Dollar bars (mismo \$ operado)
exceso de kurtosis = 4.7 (n=176)



Retornos estandarizados de SPY intradía con los dos muestreos, misma cantidad de barras. Las dollar bars (derecha) tienen menos exceso de kurtosis: al muestrear por actividad, los períodos violentos se parten en más barras y cada retorno individual es 'menos extremo'. En el Ejercicio 1 lo reproducen y testean formalmente.

§3

Las etiquetas: qué le preguntás al modelo

El fixed-horizon y sus fallas · Triple barrera · Meta-labeling

- ▶ **Para ML supervisado necesitás etiquetas: ¿esta situación fue 'compra buena' o 'compra mala'?**
- ▶ **El método que todos usan primero: fixed-horizon**
 - Mirá el retorno a N días fijos: si $r > 0 \rightarrow$ etiqueta +1; si $r < 0 \rightarrow$ etiqueta -1
- ▶ **Falla 1: ignora EL CAMINO**
 - Caíste 8% en el día 3 y recuperaste en el día 10 $\rightarrow$ etiqueta dice 'ganaste'
 - Pero en la vida real tu stop-loss te sacó en el día 3, la etiqueta describe un trade que nadie vivió
- ▶ **Falla 2: ignora la volatilidad del momento**
 - Un +1% en mercado dormido es una señal; en pleno pánico es ruido, misma etiqueta para ambos
 - Umbral fijo + vol variable = etiquetas que miden el régimen de vol, no tu señal

▶ **La idea: alrededor de cada punto de entrada, TRES barreras**

- Barrera superior (take-profit): si el precio la toca primero → etiqueta +1
- Barrera inferior (stop-loss): si la toca primero → etiqueta -1
- Barrera vertical (tiempo máximo, ej. 10 días): si se agota → etiqueta 0 (o el signo del retorno)
- La etiqueta es LA PRIMERA barrera que se toca, el evento que un trader real vive

▶ **El detalle que lo hace funcionar: barreras adaptativas a la volatilidad**

$$p^+ = p_0 (1 + k \hat{\sigma}_t) \quad p^- = p_0 (1 - k \hat{\sigma}_t)$$

- k es el multiplicador (hiperparámetro); $\hat{\sigma}_t$ = std de los últimos 20 retornos (volatilidad histórica de ventana corta)
- Se puede usar GARCH si ya está calibrado, pero vol histórica simple funciona bien: el objetivo es adaptar las barreras al régimen, no modelar la dinámica exacta de la vol
- En mercado dormido las barreras se acercan; en pánico se alejan: la MISMA pregunta en todo régimen

▶ **Triple barrera ≠ meta-labeling: dos herramientas distintas que trabajan juntas**

- Triple barrera = herramienta de ETIQUETADO: asigna +1/0/-1 a cada punto de entrada según qué barrera tocó primero
- Meta-labeling = FRAMEWORK DE DOS MODELOS: un modelo primario da la dirección, un segundo modelo (el meta-modelo) decide cuándo operar usando esas etiquetas como target

▶ Barreras asimétricas (arriba ≠ abajo) permiten expresar sesgo direccional del sistema

- ▶ **El problema: predecir la DIRECCIÓN del mercado es lo más difícil que existe**
 - Señal/ruido ínfimo (§1), pedirle eso a un solo modelo es pedirle demasiado
- ▶ **La división del trabajo**
 - MODELO PRIMARIO: decide la dirección. Puede ser simple y hasta tonto:
un cruce de medias, un factor de la clase 4, la intuición del gestor. Acierta poco más que una moneda
 - META-MODELO: NO predice el mercado, predice CUÁNDO EL PRIMARIO ACIERTA
Sus features: volatilidad del momento, régimen, fuerza de la señal, hora, liquidez...
Su etiqueta (el 'meta-label'): 1 si el primario acertó en esa observación, 0 si no
- ▶ **Por qué funciona**
 - Sube o baja? O Es este un buen momento para mi señal? no son la misma pregunta EN DIFICULTAD, aunque resolver una perfecto resuelve la otra — la ventaja de dividir el trabajo es que la pregunta del meta-modelo tiene estructura persistente (régimen) que la pregunta de dirección no tiene por sí sola
 - El meta-modelo aprende los REGÍMENES donde la señal primaria vive y donde muere

Meta-labeling II: el pipeline en cuatro etapas y el bet sizing

▶ La receta completa

- 1. Correr el modelo primario → obtener sus señales (largo/corto) en cada fecha
- 2. Etiquetar cada señal con la triple barrera → ¿el trade primario habría ganado?
- 3. Entrenar el meta-modelo (un random forest alcanza): features del momento → P(el primario acierta)
- 4. En producción: tamaño de la apuesta = dirección_primario × f(probabilidad_meta)

▶ El efecto medible: precision sube, recall baja, y eso es EXACTAMENTE lo que querés

- Sin meta: operás TODAS las señales → recall alto (no perdés ningún trade bueno), pero precision baja (muchos trades malos también)
- Con meta: operás solo en régimen favorable → precision alta (casi todos los trades que tomás son buenos), recall más bajo (te perdés algunos trades buenos que no pasaron el filtro)
- (Recall = fracción de los trades buenos históricos que efectivamente capturaste. Precision = fracción de los trades que tomaste que resultaron buenos)
- El Sharpe sube aunque aciertes menos veces EN TOTAL: apostaste grande cuando había que apostar

▶ Bet sizing: de la probabilidad al tamaño

- Criterio de Kelly calcula el tamaño de posición óptima (K%) considerando tu "ventaja" estadística

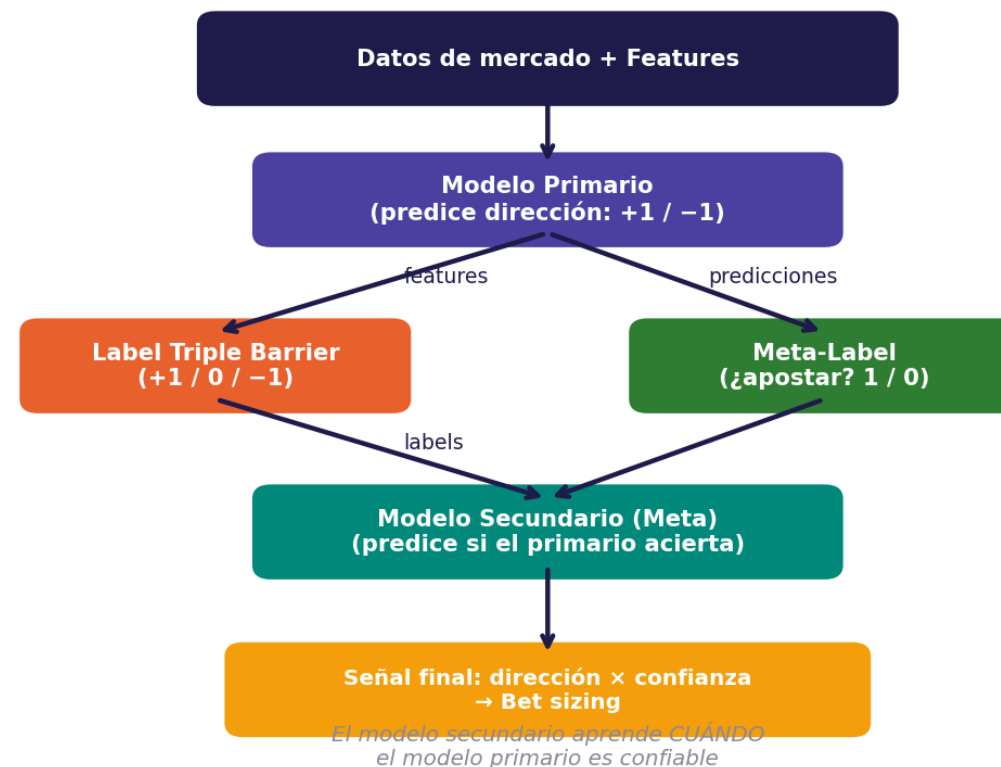
$$K \% = W - \frac{1 - W}{R}$$

W (Win rate): Es la tasa de aciertos o probabilidad de éxito de tu estrategia
R (Risk/Reward ratio): Es el ratio beneficio/riesgo promedio

Triple Barrier Method (López de Prado)
Etiquetado de observaciones



Meta-Labeling — Framework de López de Prado
(Advances in Financial Machine Learning, Cap. 3)



Izquierda: la triple barrera en acción, etiqueta +1 (toca el take-profit), -1 (toca el stop) y 0 (se agota el tiempo). Derecha: el flujo del meta-labeling, el primario pone la dirección, el meta pone la confianza, y la confianza pone el tamaño.

- ▶ **El dataset del meta-modelo tiene un problema de proporción**
 - El primario acierta poco más que una moneda. En el historial, la mayoría de las señales no fueron buenos momentos para operar:
 - → 90% de los casos: label = 0 (no era buen momento) | 10%: label = 1 (sí era buen momento)
- ▶ **El problema: un modelo que predice SIEMPRE '0' tiene 90% de accuracy**
 - Parece excelente. Pero es completamente inútil: nunca recomienda operar
 - El modelo aprendió ignorar los casos raros y adivinar siempre la clase mayoritaria
 - Es el mismo problema que en detección de fraude: los eventos raros son exactamente los que importan
- ▶ **La solución: decirle al modelo que los errores en '1' cuestan más**
 - `class_weight='balanced'` en sklearn: si el 10% son unos, penalizar un error en un '1' 9 veces más que un error en un '0'. Una línea de código, no toca los datos, no introduce leakage

§4

Los pesos: cuánta información independiente tenés

Etiquetas solapadas · Pesos por unicidad · Diferenciación fraccionaria

▶ El problema, con números

- Tus etiquetas usan ventanas de 10 días (la barrera vertical de §3)
- La etiqueta del lunes usa los días 1-10; la del martes, los días 2-11
- → comparten 9 de 10 días: son CASI LA MISMA observación, contada dos veces

▶ Por qué es grave

- Tu dataset dice 'n = 2.500 muestras'; de información independiente tenés quizás 250
- Todo lo que asume independencia se sobre-confía: los errores estándar, el bootstrap del random forest (¡muestrea con reemplazo las mismas ventanas!), los p-values, la CV
- Es el mismo pecado que el $T \geq 10N$ de la clase 5, en versión temporal

▶ La solución: pesar por unicidad

$$\tilde{u}_i = \frac{1}{|\mathcal{I}_i|} \sum_{t \in \mathcal{I}_i} \frac{1}{c_t}, \quad c_t = \#\{\text{etiquetas cuya ventana contiene } t\}$$

- Muestra que no comparte días con otras → $\tilde{u}_i = 1$; muestra que solapa todas → $\tilde{u}_i \approx 1/\text{solapamiento}$
- Se enchufa directo: `sample_weight=ũ` en sklearn, una línea, medio dataset de honestidad

▶ El dilema: precio vs retorno

- Precio ($d=0$): tiene memoria (¿estamos en máximo histórico? ¿cerca de soporte?) pero NO es estacionario → no apto para entrenar modelos ML
- Retorno ($d=1$): $r_t = P_t - P_{t-1}$. Estacionario: olvida completamente el nivel del precio
- El problema: si el nivel del precio importa para predecir (y suele importar), diferenciarlo entero borra esa información

▶ La idea: diferenciar una cantidad FRACCIONARIA $d \in (0, 1)$

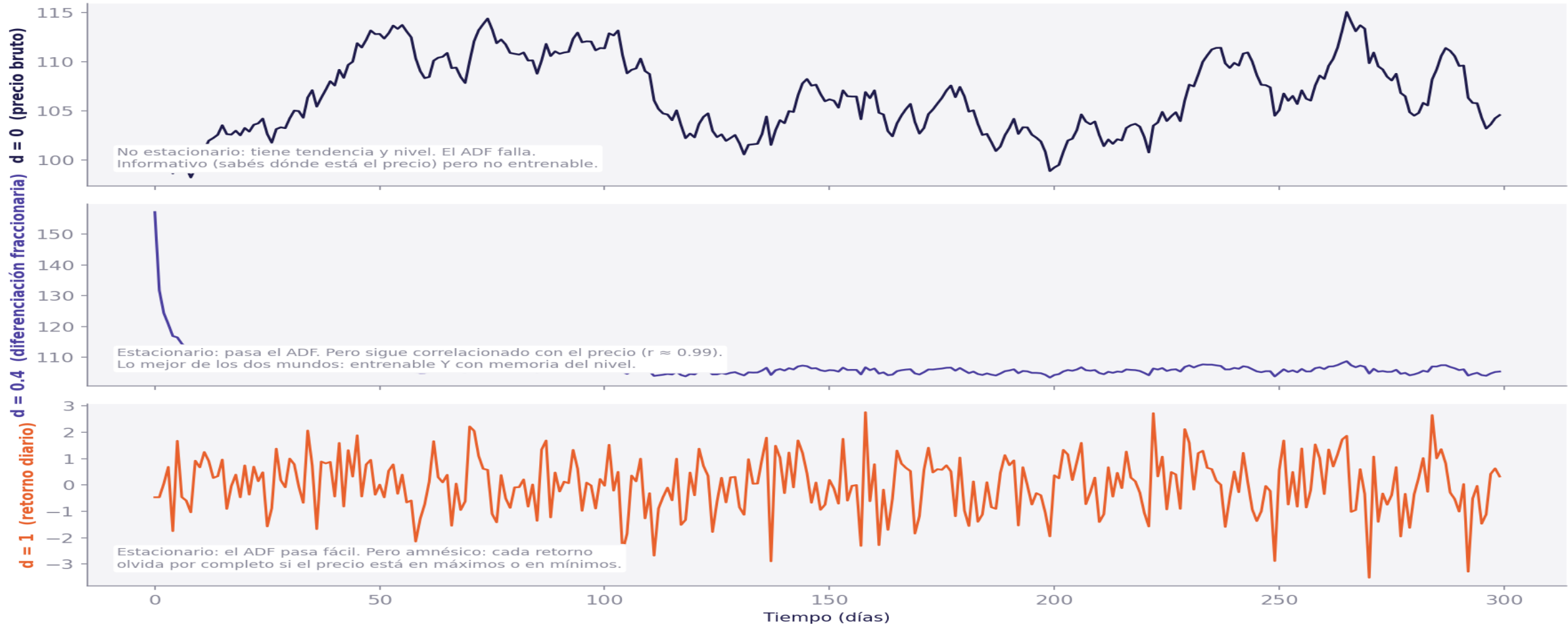
- Con $d=0.4$ (un ejemplo), la serie diferenciada pesa el pasado con pesos que decaen suavemente:
$$\tilde{X}_t \approx X_t - 0.40 X_{t-1} - 0.12 X_{t-2} - 0.06 X_{t-3} - \dots$$
- La serie recuerda el pasado, pero cada vez con menos peso: el pasado lejano casi no importa → la serie se vuelve estacionaria

▶ La receta práctica

- Buscar el d mínimo que supere el test ADF. En índices de acciones, típicamente $d^* \approx 0.35-0.5$
- Resultado: serie que pasa el ADF (entrenable) y tiene correlación >0.99 con el precio original (la memoria del nivel se preserva)

Diferenciación fraccional: el término medio entre precio y retorno

Diferenciación fraccional: el término medio entre precio y retorno



Arriba ($d=0$): el precio — no estacionario, el ADF falla, inutilizable para ML. Medio ($d=0.4$): la serie fraccional — pasa el ADF y sigue correlacionada >0.99 con el precio. Abajo ($d=1$): el retorno diario — estacionario pero amnésico, borra la memoria del nivel.

§5

La validación honesta: purged K-fold y embargo

Las dos vías del leakage · Purga · Embargo · El score que no miente

Por qué el K-fold estándar miente con series financieras

- ▶ **Recordatorio de qué hace K-fold: partir en K bloques, entrenar en K-1, testear en 1, rotar**
 - Con datos IID es perfecto. Con series de tiempo tiene DOS vías de contaminación:
- ▶ **Vía 1, el barajado temporal (la obvia)**
 - K-fold baraja: entrena con muestras de marzo para 'predecir' febrero
 - Entrenar con el futuro: el modelo ve el régimen que 'vendrá', imposible en producción
- ▶ **Vía 2, el solapamiento de etiquetas (la invisible)**
 - Tu etiqueta del 28 de enero usa la ventana hasta el 5 de febrero (barrera vertical, §3)
 - El fold de test empieza el 1 de febrero → esa muestra de ENTRENAMIENTO ya vio el test
 - Su fecha dice 'enero', pero su contenido es febrero. El leakage no está en el índice: está en la ventana
- ▶ **El resultado: scores inflados con cara de legítimos, el modelo 'funciona' hasta producción**

▶ Paso 1, PURGA

- Eliminar del ENTRENAMIENTO toda muestra cuya ventana de etiqueta se solape con el test
- Concretamente: si el test es $[t_1, t_2]$ y la etiqueta de la muestra i vive en $[t_i, t_i+h]$, se descarta i si los intervalos se intersecan

▶ Paso 2, EMBARGO

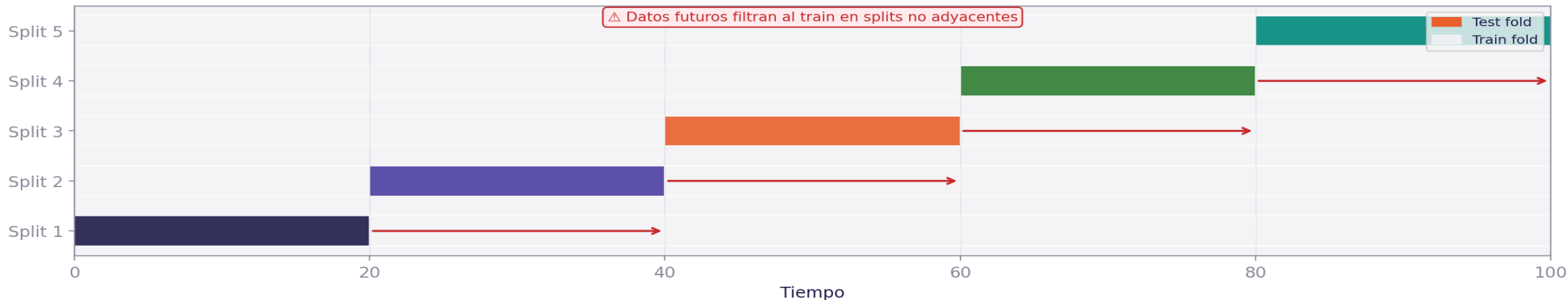
- Además, eliminar un colchón extra DESPUÉS del test (ej. 1% del dataset)
- ¿Por qué? La correlación serial: la primera muestra post-test 'recuerda' al test aunque su ventana no lo toque, el clustering de vol (clase 4) transporta información

▶ Reglas prácticas

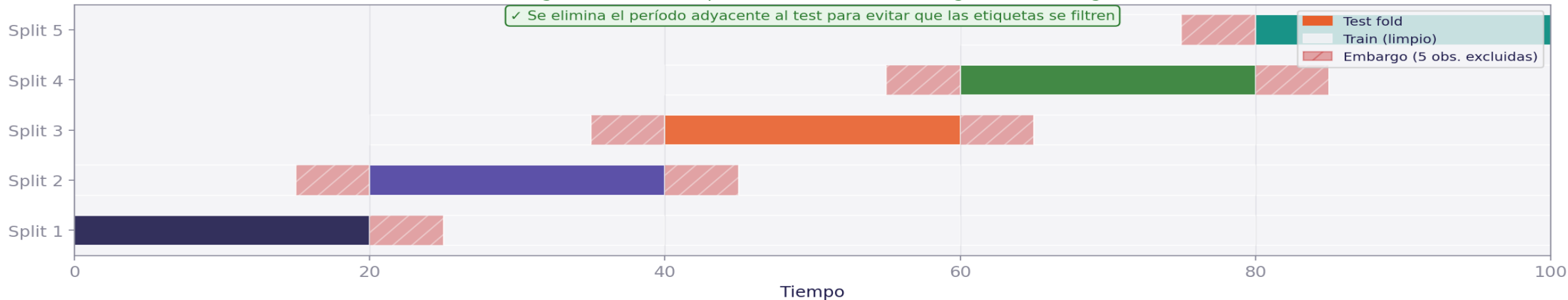
- Ventana de etiqueta h días $\rightarrow$ purga de h días y embargo $\geq h/2$ adicional
- ¿Purged K-fold o walk-forward con purging? Depende del objetivo:
- Walk-forward + purging: entrenás en el pasado, testás en el futuro — replica exactamente la producción. El más honesto para la evaluación final. Costo: los últimos datos casi no entrenan, estimaciones más variables
- Purged K-fold: usa todos los datos (incluso futuros en train), más eficiente. Útil para seleccionar modelos e hiperparámetros, pero no simula producción exactamente
- En la práctica: purged K-fold para elegir el modelo, walk-forward + purging para el veredicto final antes de producción

Purged K-fold vs K-fold estándar

K-Fold estándar — LEAKAGE: el test set está mezclado con el train



Purged K-Fold CV (López de Prado) — sin leakage, con embargo



Arriba: K-fold estándar, las etiquetas cuya ventana cruza la frontera del test contaminan el entrenamiento. Abajo: purged K-fold, se eliminan las muestras solapadas (purga) y un colchón adicional (embargo, rayado) antes y después del test.

§6

El modelo y sus features: qué aprendió de verdad

Bagging vs boosting (XGBoost) · MDI · MDA · La trampa de la sustitución

- ▶ **Familia lineal — frontera de decisión lineal en el espacio de features**
 - Regresión logística: $P(y=1|x) = \sigma(w'x + b)$. Interpretable, probabilístico, raramente sobreajusta.
 - En finanzas: el baseline obligado. Si la señal es débil y lineal, ningún modelo complejo la supera
- ▶ **Familia de árboles — captura no-linealidades e interacciones sin asumir forma funcional**
 - Random Forest: B árboles en PARALELO sobre remuestreos bootstrap → promedio reduce varianza
En finanzas: robusto al ruido, maneja features mixtos, produce importancias (MDI/MDA, ver §6)
 - XGBoost / LightGBM: árboles en SECUENCIA, cada uno corrige los residuos del anterior (gradiente)
En finanzas: state of the art en datos tabulares; LightGBM es 5-20x más rápido (crecimiento hoja-por-hoja)
- ▶ **Familia de redes — aprenden sus propias representaciones de la secuencia cruda**
 - LSTM: red recurrente con compuertas que aprenden qué recordar y qué olvidar a lo largo de la secuencia
 - CNN-1D: desliza 'detectores de patrones' aprendidos sobre la serie — captura estructuras locales precio/volumen
 - En finanzas: útiles en alta frecuencia y LOB donde hay muchos datos y patrones locales fuertes

- ▶ **La idea: transformar una puntuación lineal en una probabilidad entre 0 y 1**

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

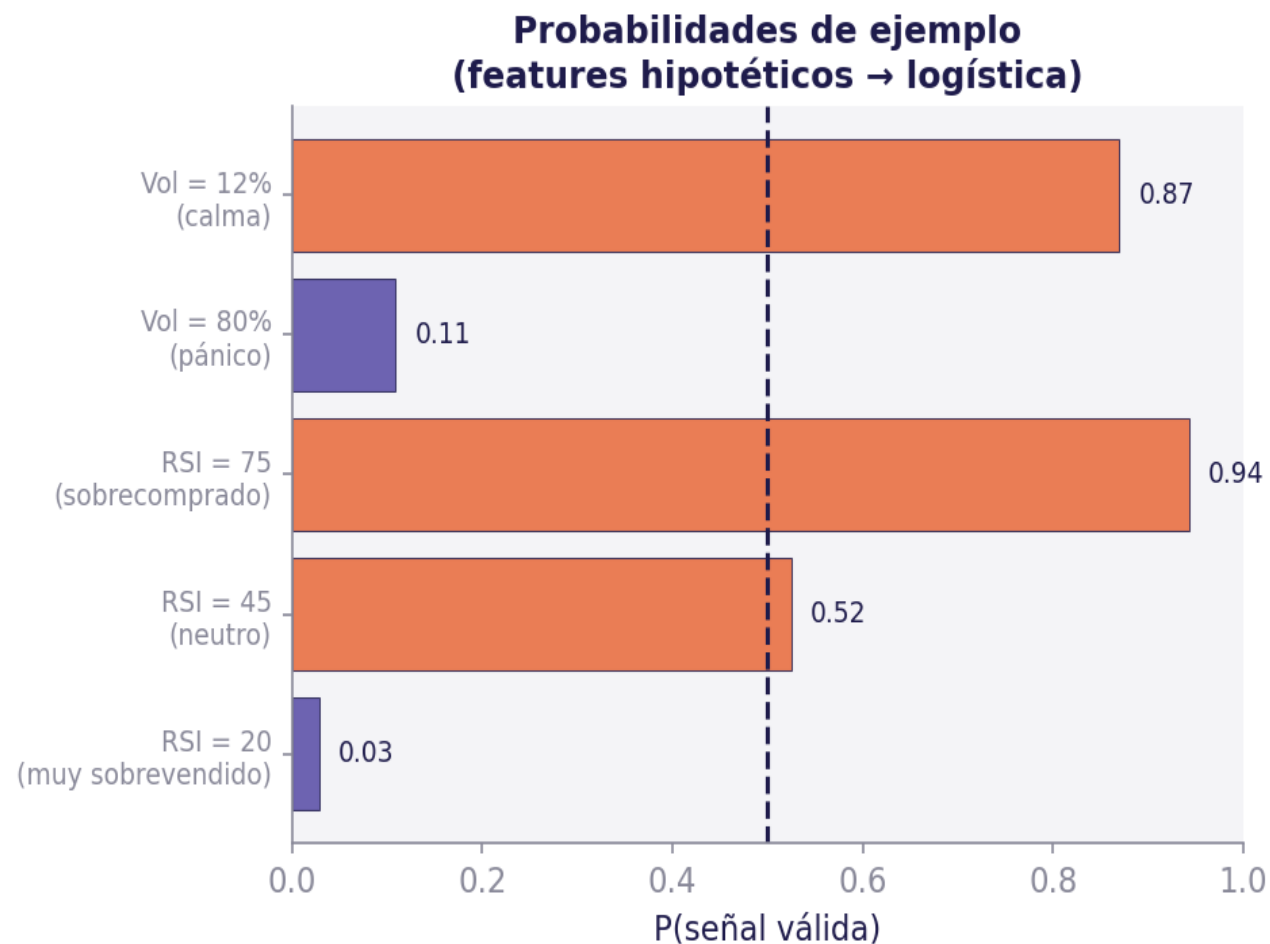
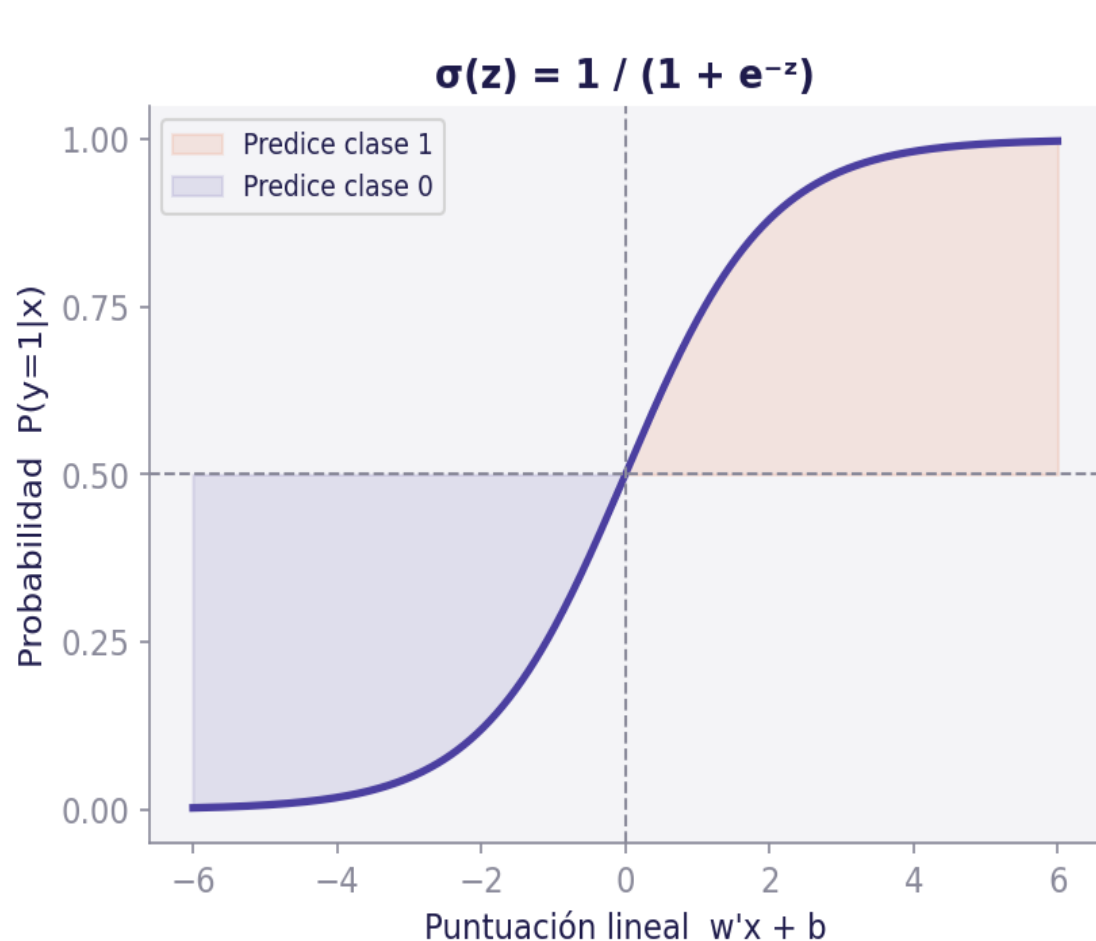
- Parámetros aprendibles: w (un peso por feature) y b (el sesgo) — los que definen la fórmula de arriba

- ▶ **Cómo se ajusta**

- El entrenamiento busca los w y b que hacen que las probabilidades predichas se parezcan lo más posible a lo que realmente pasó en los datos históricos.
- λ es la MISMA penalización de Lasso (clase 5), $\lambda \|\mathbf{w}\|_1$, sumada a lo que se minimiza para elegir w — no es 'correr un Lasso aparte', es incorporar esa idea dentro del ajuste de la logística. Con $\lambda > 0$ empuja pesos irrelevantes exactamente a cero: selección automática de features

- ▶ **Por qué es la primera línea de defensa en finanzas**

- Interpretable: $w_j > 0 \rightarrow$ la feature j empuja la probabilidad hacia arriba. La salida $P(y=1|x)$ es bet-sizing directamente



Izquierda: la función sigmoide transforma cualquier puntuación lineal $w'x+b$ en una probabilidad entre 0 y 1. El umbral en $p=0.5$ decide la clase. Derecha: ejemplos financieros — a baja volatilidad el modelo asigna alta probabilidad de señal válida (0.87); en pánico ($\text{vol}=80\%$) cae a 0.11; un RSI muy sobrevendido (20) da solo 0.03.

Cuándo conviene

- ▶ La señal es débil y (sospechosamente) lineal — es el default seguro, la vara que cualquier otro modelo tiene que superar
- ▶ Necesitás interpretabilidad: cada peso w_j te dice, directo, si esa feature empuja la probabilidad hacia arriba o abajo

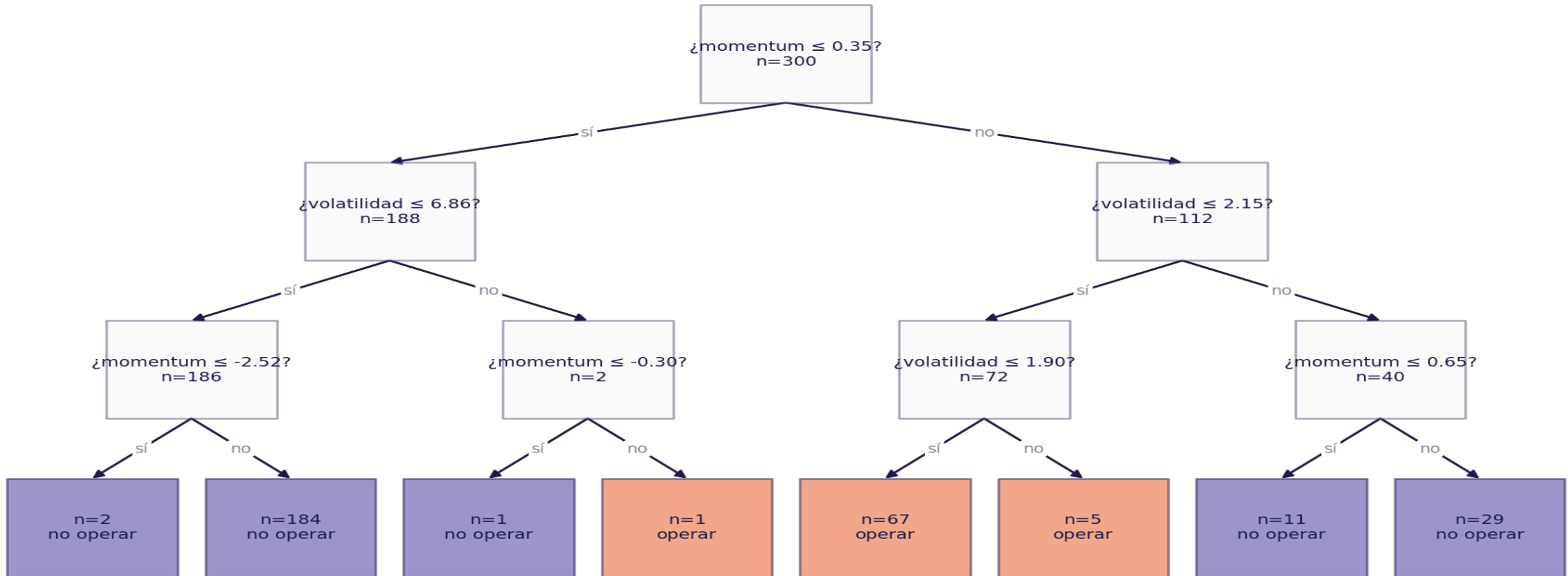
El costo que pagás

- ▶ Se queda corta si la relación real es no-lineal — no hay forma de estirla más allá de un hiperplano
- ▶ No captura interacciones entre features a menos que las agregues vos a mano (ej. momentum $\times$ volatilidad)

- ▶ **Un árbol predice con una cadena de preguntas binarias: '¿momentum > 0.3?', y así sucesivamente**
 - Cada pregunta parte los datos en dos; al final de cada rama, una HOJA predice el promedio o la clase mayoritaria de lo que cayó ahí
- ▶ **Los hiperparámetros que importan**
 - max_depth: cuántas preguntas seguidas — chico = underfit, grande = memoriza el ruido (overfit)
 - min_samples_leaf: mínimo de datos por hoja — subirlo es la forma más simple de frenar el sobreajuste

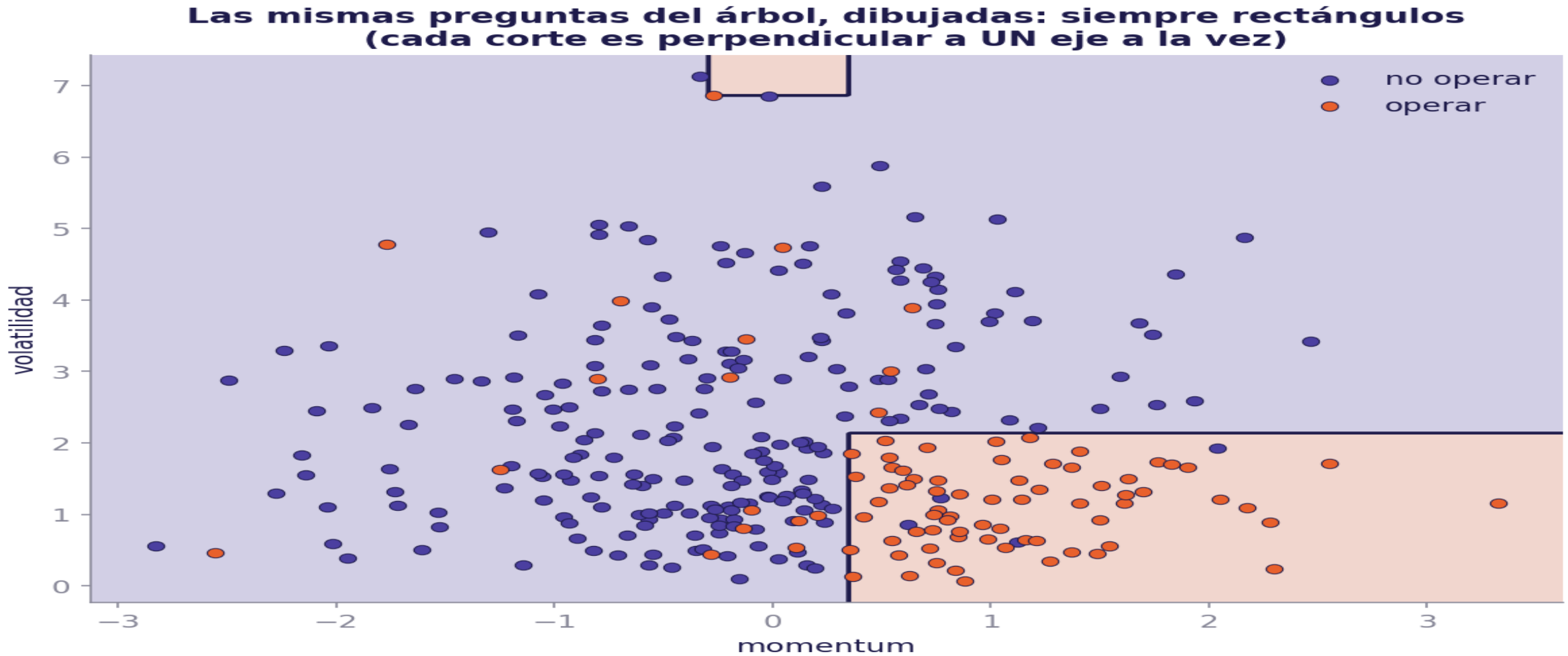
Un árbol, por dentro: una cadena de preguntas

Un árbol es una cadena de preguntas del tipo " $\text{feature} \leq \text{umbral?}$ "



Un árbol de profundidad 3 sobre dos features sintéticos (momentum, volatilidad). Cada nodo pregunta ' $\text{feature} \leq \text{umbral?}$ '; True va a la izquierda, False a la derecha. Las hojas violetas predicen 'no operar', las naranjas 'operar'.

Las preguntas del árbol, dibujadas: siempre rectángulos



Cada pregunta agrega un corte perpendicular a UN eje (momentum o volatilidad), nunca diagonal. El resultado es siempre un mosaico de rectángulos — la limitación que las redes neuronales (más adelante) no tienen.

Random Forest vs Gradient Boosting

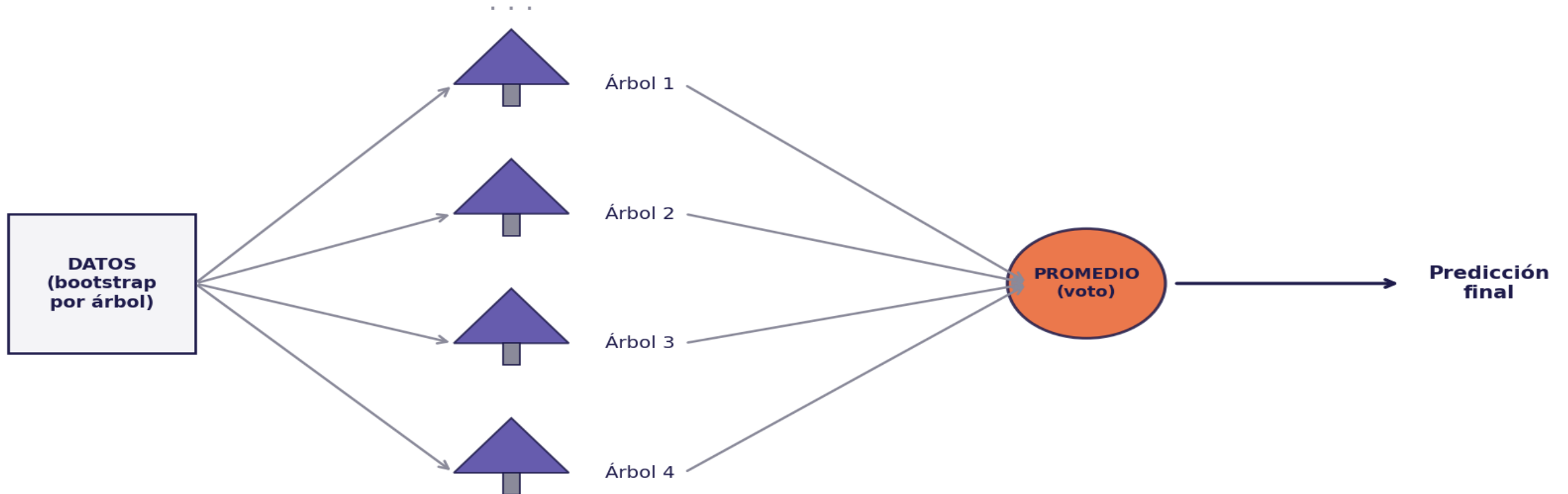
Random Forest — muchos expertos en paralelo

- ▶ La idea: B árboles independientes, cada uno con su propia muestra bootstrap; se promedian al final
- ▶ Hiperparámetros: `n_estimators` (más es casi gratis), `max_features` ($\sqrt{p}$), `max_depth`
- ▶ Ventaja: robusto, difícil de arruinar, poco tuneo. Desventaja: rara vez es 'el mejor' — pero rara vez el peor

Gradient Boosting — una cadena que corrige errores

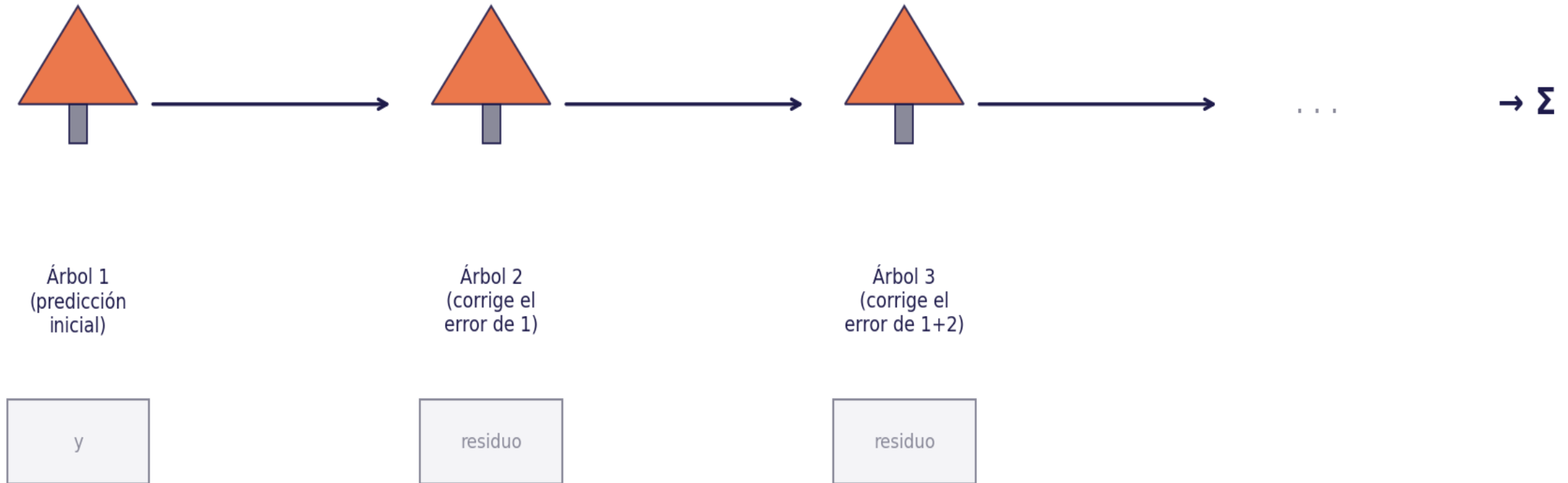
- ▶ La idea: árboles en secuencia, cada uno aprende el residuo (error) de los anteriores; se suman
- ▶ Hiperparámetros: `learning_rate` (η), `n_estimators` + early stopping, `max_depth` chico
- ▶ Ventaja: puede ganarle a RF si la señal es real y no-lineal. Desventaja: el que más explota leakage si la CV no es honesta (§5)

Random Forest: B árboles EN PARALELO, cada uno con su propio remuestreo, se combinan al final por promedio (o voto) — nunca se miran entre sí



B árboles en paralelo, cada uno con su propio remuestreo — se combinan recién al final por promedio o voto. Nunca se miran entre sí durante el entrenamiento.

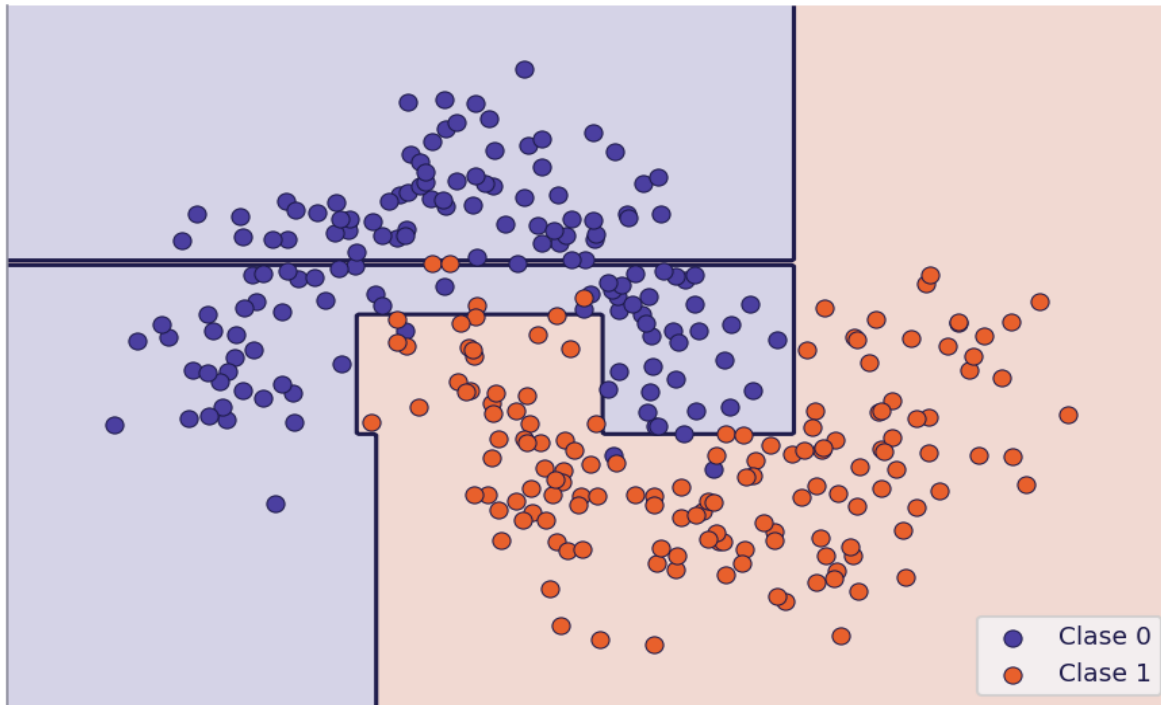
Gradient Boosting: árboles EN SECUENCIA — cada uno aprende SOLO lo que el anterior se equivocó (el residuo), la predicción final es la suma de todos



Árboles en secuencia: cada uno aprende solo el error que dejaron los anteriores. La predicción final es la suma de toda la cadena.

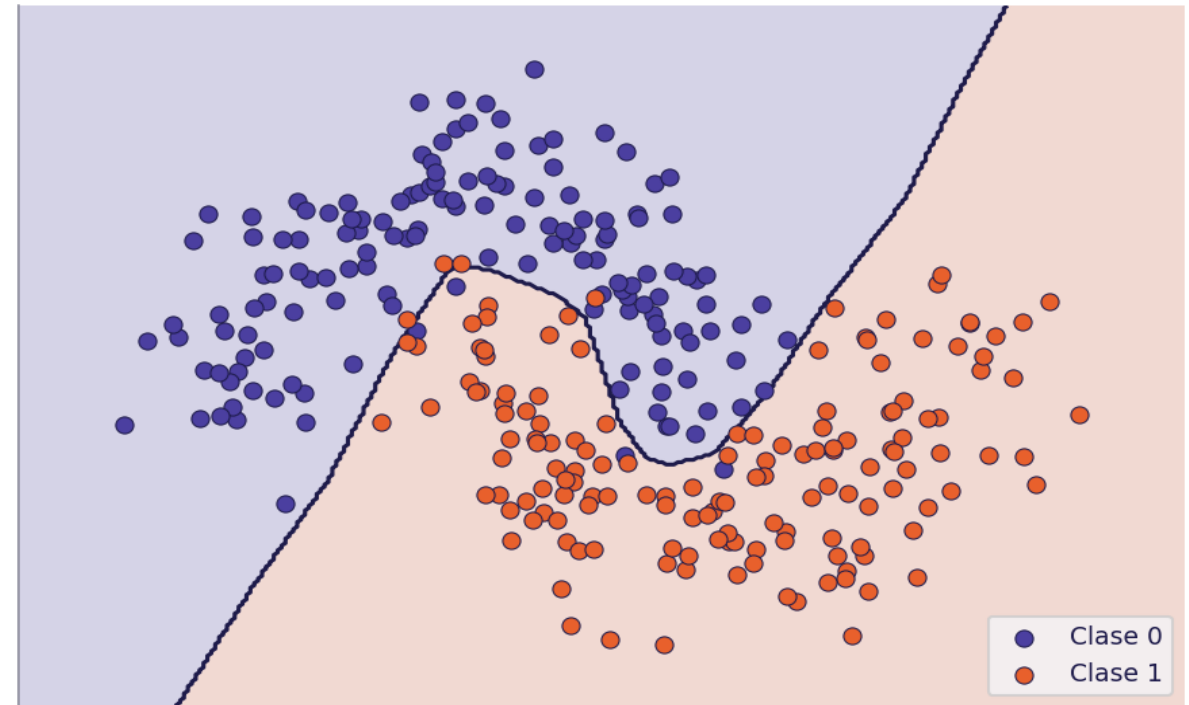
La misma clasificación, dos familias de modelos

Árbol de decisión (Random Forest)



Divide el espacio en RECTÁNGULOS
con cortes perpendiculares a los ejes

Red neuronal (MLP / LSTM / CNN)



Aprende fronteras CURVAS y arbitrarias
con cualquier forma

Mismo dataset (*make_moons*, dos clases entrelazadas). Izquierda — árbol de decisión: talla el espacio en rectángulos paralelos a los ejes (*splits en un feature a la vez*). Derecha — red neuronal: frontera suave y curva que sigue la forma real de los datos. En finanzas esto importa: si la señal es una interacción no-axial entre features, un árbol la particionará groseramente; si los datos son escasos, la red se sobreajusta rápido.

¿Qué es una red neuronal? Concepto e hiperparámetros

▶ Una red apila capas: cada una combina los datos linealmente y les aplica una no-linealidad ϕ

$$h = \phi(Wx + b)$$

- Apiladas muchas veces, van 'doblando' el espacio hasta que una frontera curva se vuelve separable — por eso traza curvas en vez de los rectángulos de un árbol
- ¿Para qué sirve ϕ ? Sin ella, apilar capas lineales seguiría siendo una transformación lineal (componer matrices da otra matriz) — ϕ es lo que le da a la red su poder real de curvar el espacio (fórmula y las tres más usadas, próxima slide)

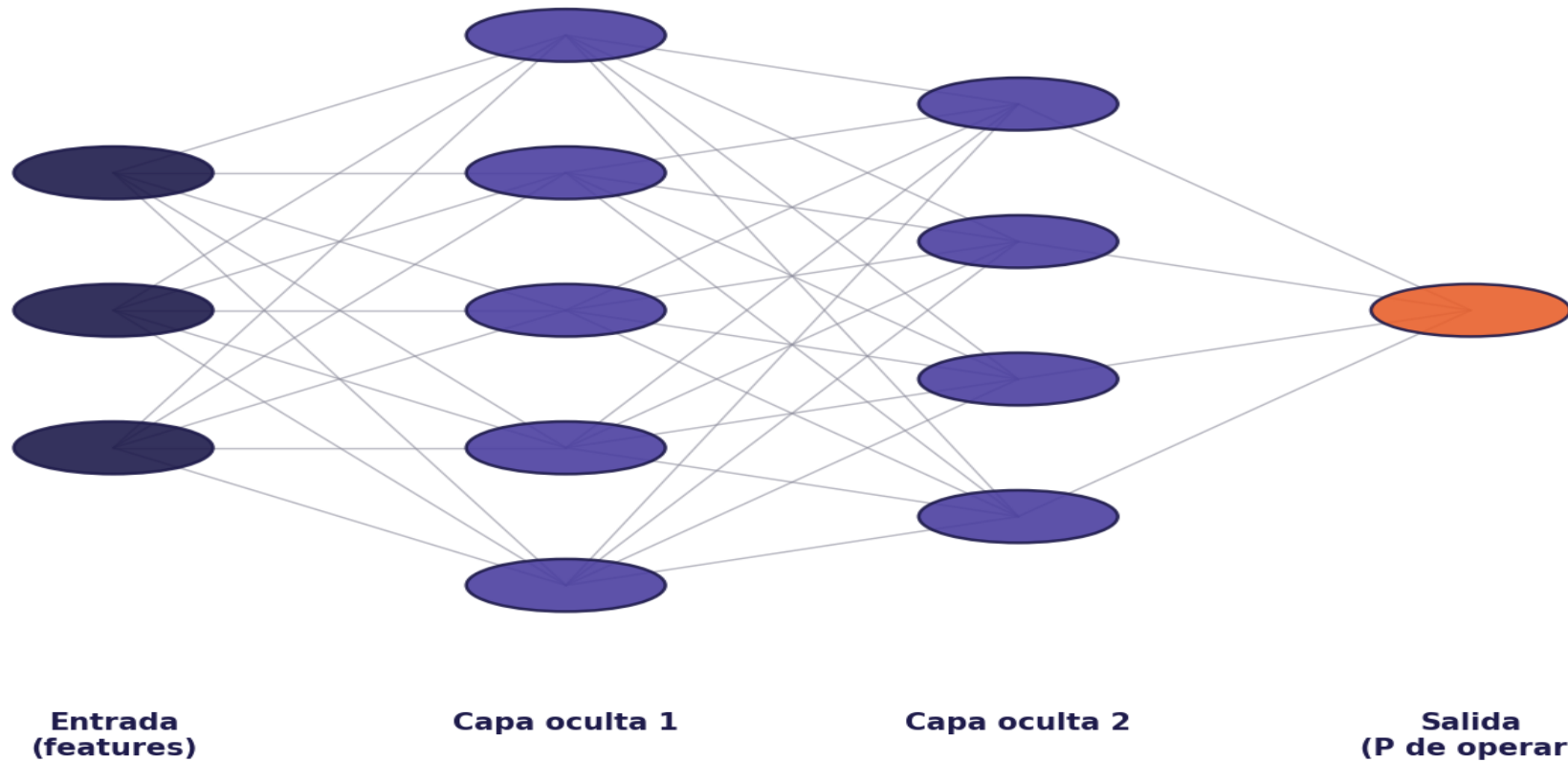
▶ Hiperparámetros principales

- Arquitectura: profundidad (capas) y ancho (neuronas por capa) — más de ambas = más capacidad, para señal Y ruido
 - learning_rate: el tamaño del paso al entrenar — el más sensible de todos
 - Regularización: dropout y weight decay — las versiones-red de la λ que ya vieron en Lasso/Ridge
- ▶ Ventaja: la máxima flexibilidad posible. Desventaja: la más fácil de sobreajustar con pocos datos, y la menos interpretable

Una red neuronal es una cadena de "combinación lineal + no-linealidad" apilada capa tras capa — cada línea es un peso que se aprende

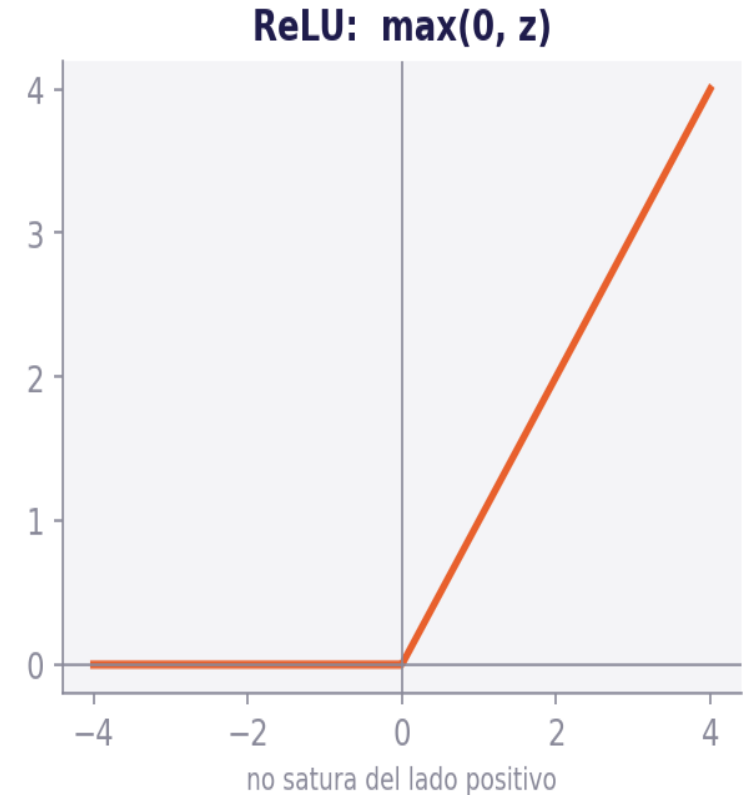
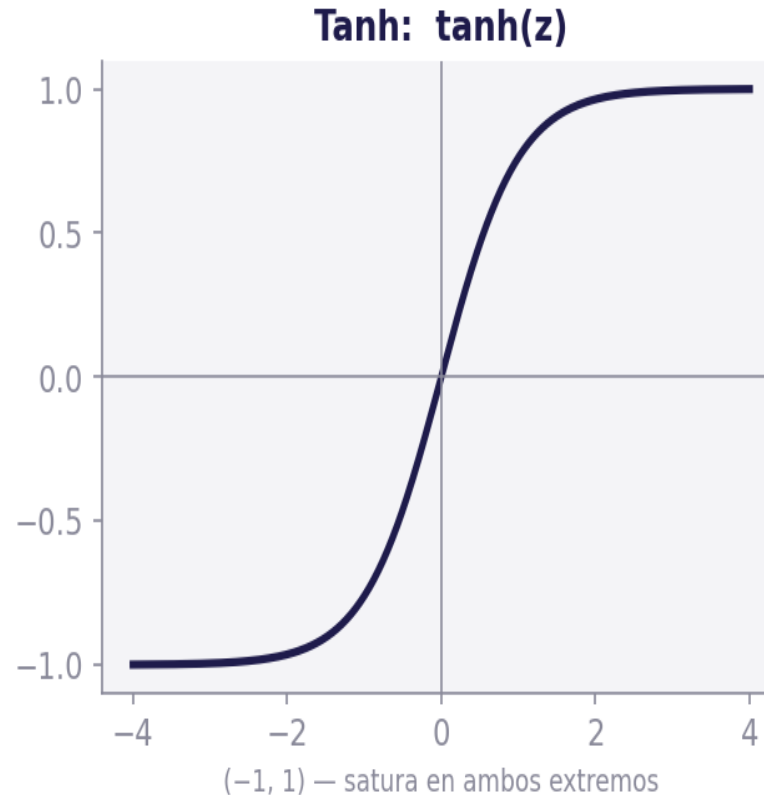
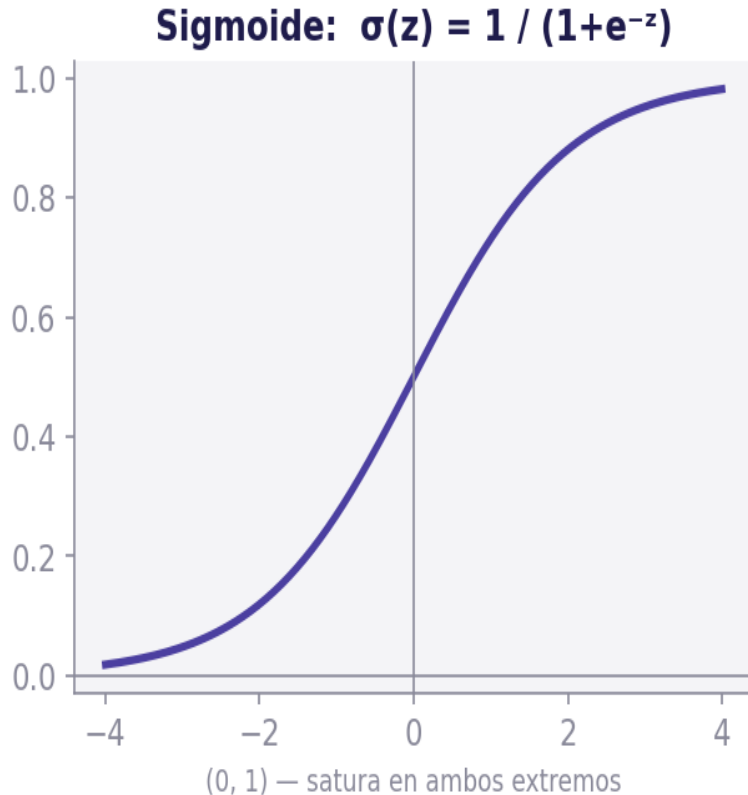
$$h_1 = \phi(W_1x + b_1)$$

$$h_2 = \phi(W_2h_1 + b_2)$$



Cada nodo es una neurona, cada línea un peso que se aprende. La entrada recibe los features, cada capa oculta aplica $h = \phi(Wh_{\text{anterior}} + b)$, y la salida da la probabilidad de operar.

ϕ : la no-linealidad que se aplica después de cada combinación lineal



Sigmoide: aplasta todo a (0,1) — útil en la capa de SALIDA cuando se quiere una probabilidad directamente. Tanh: aplasta a (-1,1), centrada en 0 — usada históricamente en capas ocultas. ReLU: $\max(0,z)$ — hoy el default en capas ocultas, porque no 'satura' del lado positivo (el gradiente no se apaga ahí). Cuando sigmoide/tanh Sí saturan, su derivada es ≈ 0 y el gradiente se diluye capa a capa — el problema que LSTM resuelve con compuertas (próximas slides).

¿y si le damos al modelo la máxima flexibilidad posible?

- ▶ **Random forest y XGBoost son poderosos, pero trabajan sobre features que NOSOTROS diseñamos (vol, momentum, RSI...)**
- ▶ **Deep Learning promete algo distinto: darle la serie CRUDA y que la red aprenda sus PROPIOS patrones**
 - Dos arquitecturas hechas a medida para secuencias: LSTM (redes recurrentes) y CNN-1D (convoluciones)
 - Recurrente: procesa la secuencia paso a paso; el estado que calcula en cada paso (memoria de lo visto hasta ahí) vuelve a entrar como insumo del paso siguiente — a diferencia de una red normal, que mira todo el input de una sola vez y no arrastra nada de un dato al otro
 - Convolución: desliza una ventana de pesos sobre la serie, calculando un promedio ponderado en cada posición — la misma cuenta que un promedio móvil, pero con pesos APRENDIDOS en vez de fijados a mano

- ▶ **La idea: procesa la secuencia paso a paso, manteniendo un 'estado interno' (memoria)**
 - El problema de una red recurrente simple: esa memoria se 'olvida' muy rápido (el gradiente se diluye a cada paso)
 - LSTM lo resuelve con compuertas APRENDIBLES: qué olvidar, qué agregar, qué mostrar en cada paso
 - Hiperparámetro extra (sobre los de cualquier red): el tamaño del estado de memoria
- ▶ Ventaja: no hace falta diseñar features a mano, la red mira la secuencia cruda. Desventaja: necesita MUCHOS más datos que un RF para entrenarse bien

CNN-1D: ¿es como ARIMA pero más complejo?

- ▶ **Casi: ambos aplican una ventana deslizante sobre la serie — un promedio ponderado de los últimos K valores**
 - La diferencia: en CNN esos pesos se APRENDEN de los datos (no se fijan por un modelo teórico), hay MUCHOS 'kernels' en paralelo (cada uno detecta un patrón distinto), y las capas se apilan detectando patrones cada vez más abstractos
 - En una frase: ARIMA es una CNN con un solo kernel fijo y sin capas

Lo que sí cambia entre modelos

- ▶ Interpretabilidad: logística > árboles > redes — cuánto podés explicar una decisión individual
- ▶ Costo de entrenar y tunear: logística, muy barato — RF, barato — GBT, medio — LSTM/CNN, caro
- ▶ Capacidad de capturar no-linealidades: logística < árboles < redes

Lo que NO cambia (la tesis de toda la clase)

- ▶ Con señal/ruido tan bajo como en finanzas, más capacidad no ataca el problema de fondo — solo memoriza mejor el ruido
- ▶ El 'mejor modelo' depende de la VALIDACIÓN, no de la arquitectura: con CV ingenua gana el más complejo, con purged CV casi ninguno gana de verdad
- ▶ Por eso LDP lo dice sin vueltas: sofisticar el modelo es lo ÚLTIMO que hay que hacer — primero datos, etiquetas y validación honesta

\$7

El backtest que miente: Deflated Sharpe y PBO

El máximo del ruido · La corrección por intentos · Higiene de laboratorio

El problema del Sharpe selectivo: cuando el mejor ruido parece señal

► **Imaginá este experimento: tirás 200 monedas durante 5 años y elegís la que más veces dio cara**

- Cada moneda tiene Sharpe esperado = 0. Pero la mejor de 200 va a parecer que 'funciona' — simplemente porque entre tantas, alguna tuvo suerte por rachas largas
- Esto es exactamente lo que hacés cuando probás 200 variantes de tu estrategia y te quedás con la mejor

► **¿Cuánto Sharpe puede parecer tener la mejor estrategia por pura suerte?**

$$SR_0 = \mathbb{E} \left[\max_N SR_{\text{ruido}} \right] \approx \sqrt{\frac{2 \ln N}{T}}$$

- A ESO lo llamamos SR_0 : el 'Sharpe de ruido' — el mejor Sharpe que el puro azar puede fabricar, sin que exista ninguna señal real, con $N=200$ estrategias en 5 años: $SR_0 \approx 0.7-1.0$
- Tu modelo reporta Sharpe=0.8? Si probaste 200 variantes, SR_0 ya es 0.8. No hay evidencia de señal real

► **¿Qué cuenta como un intento? Todo lo que hiciste mirando el resultado**

- Probaste learning rate 0.01, 0.05 y 0.1 → +3. Cambiaste el umbral de la triple barrera → +1. Agregaste momentum como feature y lo dejaste porque mejoró → +1. Corriste sobre SPY y QQQ y te quedaste con el mejor → +2. Cambiaste el período de train porque 'el mercado cambió' → +1

► **La idea es simple: ¿tu Sharpe es significativamente mayor que SR_0 , el Sharpe de ruido de la slide anterior?**

- Contra un benchmark que no es cero: ¿el exceso ($\hat{SR} - SR_0$) es grande relativo al error de estimación del Sharpe?

$$DSR = \Phi\left(\frac{(\hat{SR} - SR_0)\sqrt{T-1}}{\sqrt{1 - \gamma_3\hat{SR} + \frac{\gamma_4-1}{4}\hat{SR}^2}}\right)$$

- Numerador: $\hat{SR} - SR_0$. Cuánto te alejás del mejor ruido. El denominador es el error del Sharpe, ajustado por la no-normalidad de los retornos (skewness γ_3 y exceso de curtosis γ_4 penalizan, porque colas gordas inflan el Sharpe calculado)
- $\Phi(\cdot)$ es la normal acumulada: $DSR \in (0,1)$

► **Cómo interpretar DSR**

- DSR es la probabilidad de que tu estrategia sea genuina, dado cuántas cosas probaste y la distribución de los retornos
- $DSR > 0.95$: el resultado es defendible. $DSR \approx 0.5$: sos una moneda. $DSR < 0.5$: peor que el ruido esperado

▶ **PBO, Probability of Backtest Overfitting: la otra herramienta, con una pregunta muy simple**

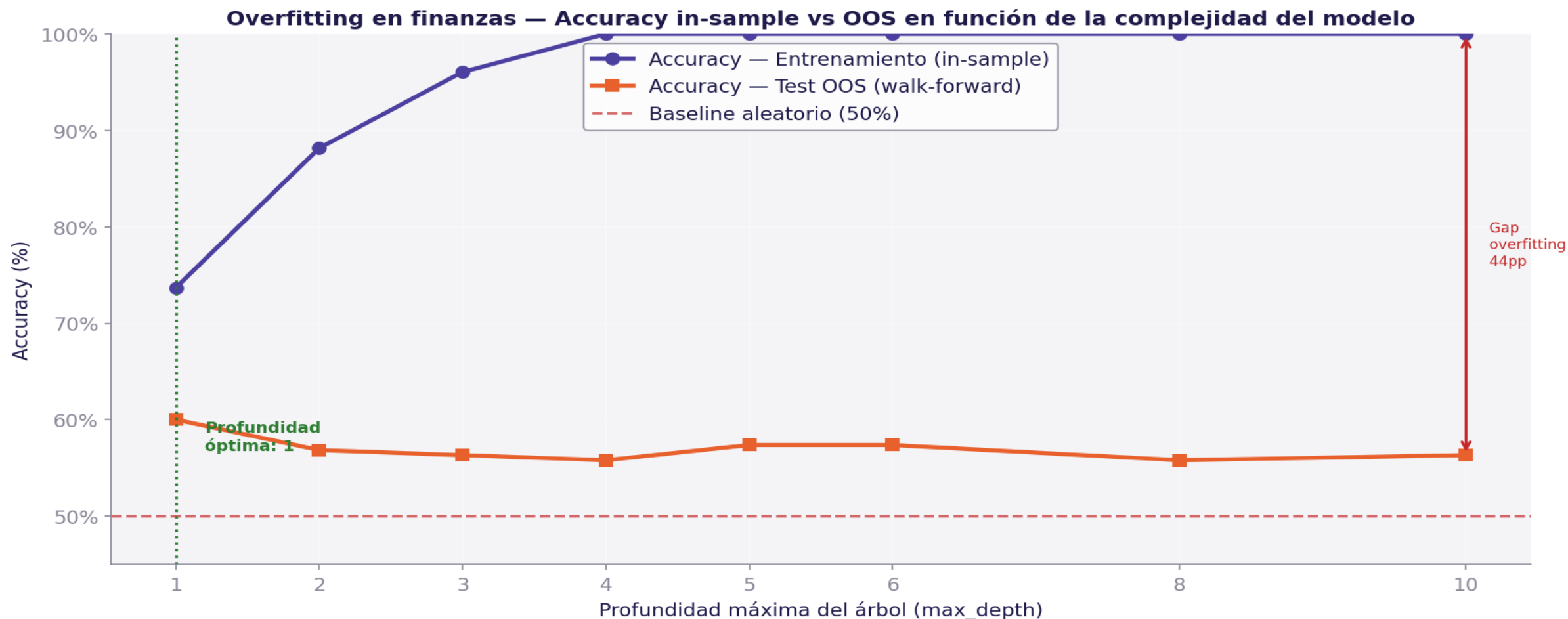
- Elegiste 'la mejor' estrategia mirando UNA mitad de los datos (in-sample). Si hubieras mirado la OTRA mitad en cambio, ¿seguiría siendo la mejor?
- PBO repite ese experimento muchas veces, partiendo la historia de formas distintas (todas las combinaciones posibles de mitades), y cuenta: ¿qué fracción de las veces tu 'campeona' de una mitad resultó ser de las PEORES en la otra?
- $PBO \approx 50\%$ → elegir tu estrategia ganadora es lo mismo que tirar una moneda: tu proceso de selección no encontró señal real, encontró suerte

▶ **El checklist**

- ✓ Registrar TODOS los experimentos (el N del DSR es el N verdadero, no el confesable)
- ✓ Features con datos hasta t, nunca t+1 (ni en la normalización: medias del futuro son leakage)
- ✓ Purged CV con embargo $\geq$ ventana de etiqueta, siempre
- ✓ Costos de transacción realistas IMPORTANTE
- ✓ El dataset de test final se toca UNA vez, es un recurso no renovable

▶ El backtest no es un experimento repetible: hay UNA historia, gastarla con respeto

Por qué hace falta todo esto: la raíz del problema, en una curva



Esta curva es la explicación visual de por qué existen el DSR y el PBO: muestra a ojo la misma enfermedad que ellos miden con un número. Eje X: complejidad del modelo (más árboles, más profundidad, más parámetros — o, para el DSR, más intentos probados). Eje Y: error. El error de train siempre baja con más complejidad (memorización); el de test baja al principio y VUELVE A SUBIR (empieza a memorizar ruido). En finanzas ese mínimo está MUY a la izquierda de lo que otros dominios enseñan a esperar — hay tan poca señal que modelos simples ganan. El DSR y el PBO son la versión cuantificada de 'a la derecha de este mínimo, estás mintiéndote'.

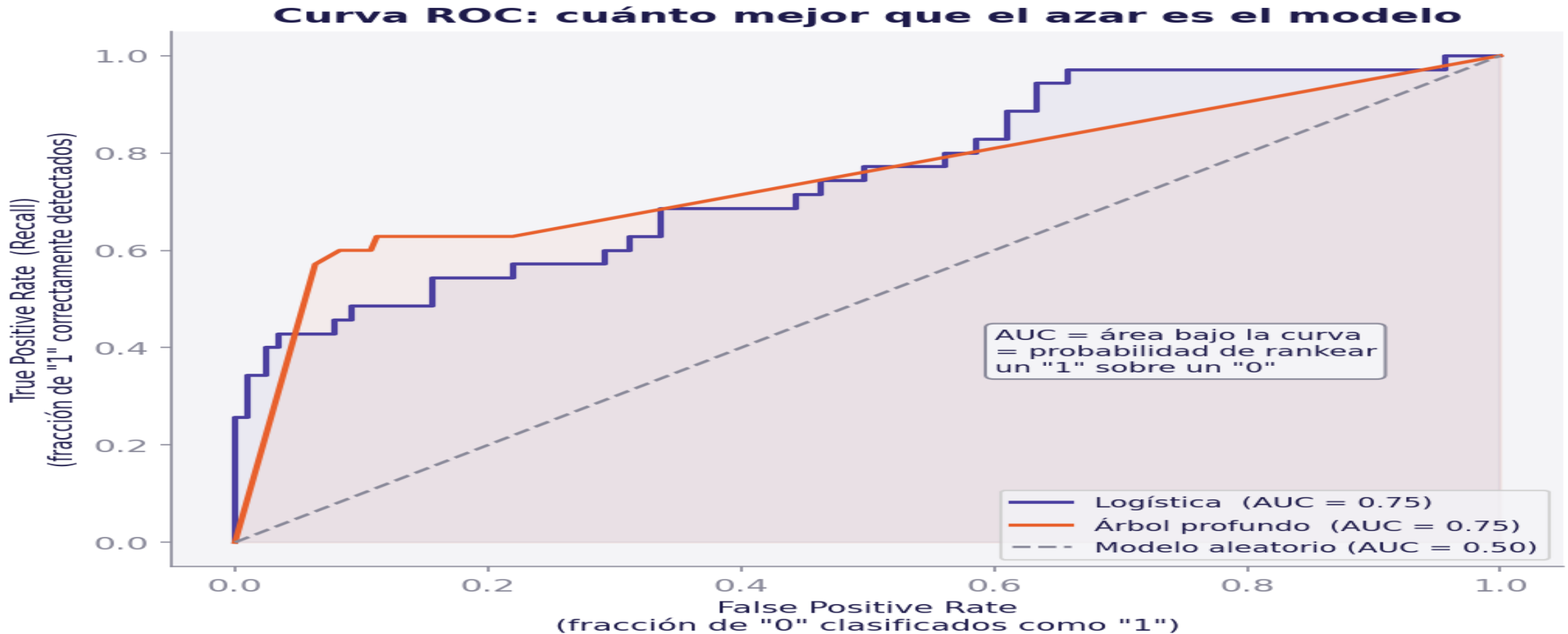
§8

Herramientas de evaluación y cierre

AUC · Curva de aprendizaje · El pipeline completo

- ▶ **El problema con accuracy:** si el 90% de las muestras son clase 0, un modelo que siempre predice 0 tiene 90% de accuracy. Inútil (lo vimos en la slide 19)
- ▶ **AUC-ROC (Area Under the ROC Curve): la alternativa estándar**
 - La curva ROC grafica, para cada umbral de clasificación posible:
 - Eje Y: True Positive Rate = fracción de los '1' reales que el modelo detectó (= recall)
 - Eje X: False Positive Rate = fracción de los '0' reales que el modelo clasificó mal como '1'
 - Al subir el umbral, el modelo es más conservador: detecta menos '1' reales pero también genera menos falsas alarmas. La curva traza ese trade-off
- ▶ **El AUC es el área bajo esa curva. Interpretación directa:**
 - $AUC = 1.0$: el modelo es perfecto. $AUC = 0.5$: equivale a tirar una moneda. $AUC = 0.6$ en finanzas ya es un resultado interesante
 - Ventaja clave: no depende del umbral de clasificación ni del balance de clases. Por eso es preferible a accuracy cuando los eventos son raros (señales de trading, fraude)

Curva ROC: cuánto mejor que el azar es el modelo



Cada punto de la curva corresponde a un umbral de clasificación distinto. La diagonal es el modelo aleatorio (AUC=0.50). Logística y árbol profundo ambos alcanzan AUC=0.75, pero con forma diferente: la logística mejora suavemente el recall; el árbol tiene un salto inicial brusco. En producción, AUC=0.6 ya es considerado interesante en finanzas.

Curva de aprendizaje: ¿necesito más datos o un mejor modelo?

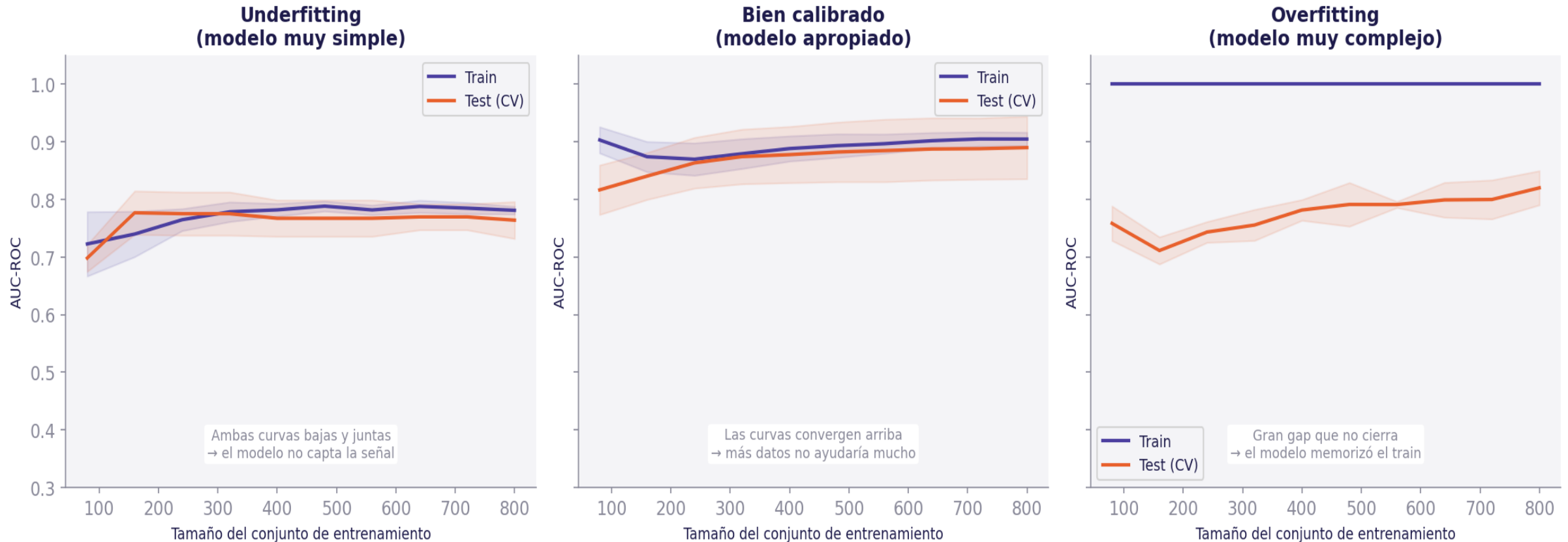
- ▶ **Es el score del modelo (train y test) graficado contra el TAMAÑO del conjunto de entrenamiento**
 - Aquí se varía la cantidad de datos manteniendo el modelo fijo

- ▶ **Tres diagnósticos que da la curva de aprendizaje**
 - 1. Score de train y test convergen ARRIBA (ambos altos): el modelo está bien. Agregar más datos no ayuda mucho
 - 2. Score de train y test convergen ABAJO (ambos bajos): underfitting. El modelo es demasiado simple para capturar la señal. Solución: más features, más complejidad
 - 3. Gran GAP entre train (alto) y test (bajo) que no cierra con más datos: overfitting estructural. Solución: regularizar más, simplificar el modelo

- ▶ **En finanzas la curva de aprendizaje casi siempre muestra el caso 3: el gap no cierra**
 - La razón: la señal/ruido es tan baja que el modelo encuentra patrones espurios en cualquier muestra finita. Agregar más datos del mismo período no resuelve el problema de fondo — cambia el régimen

Curva de aprendizaje: tres diagnósticos en un solo gráfico

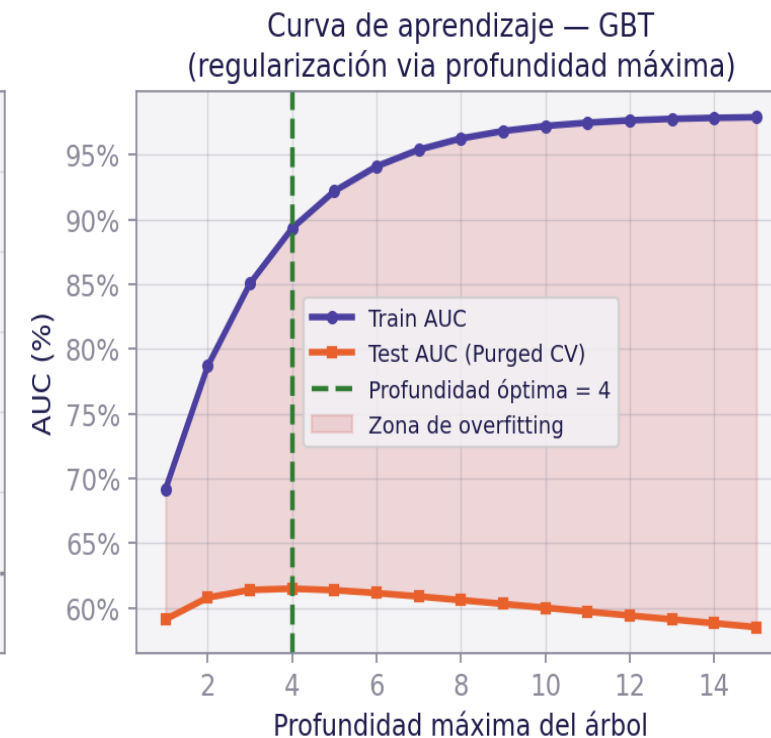
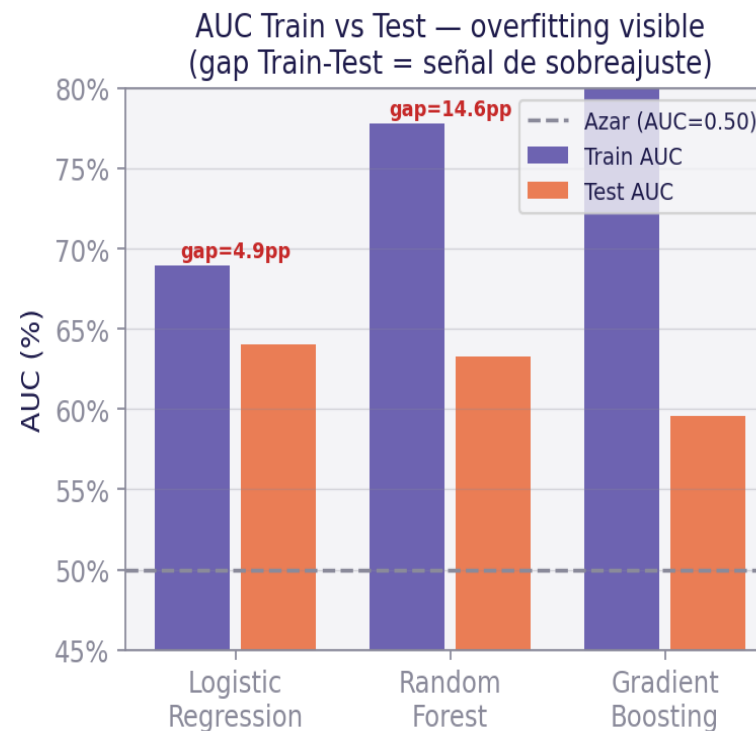
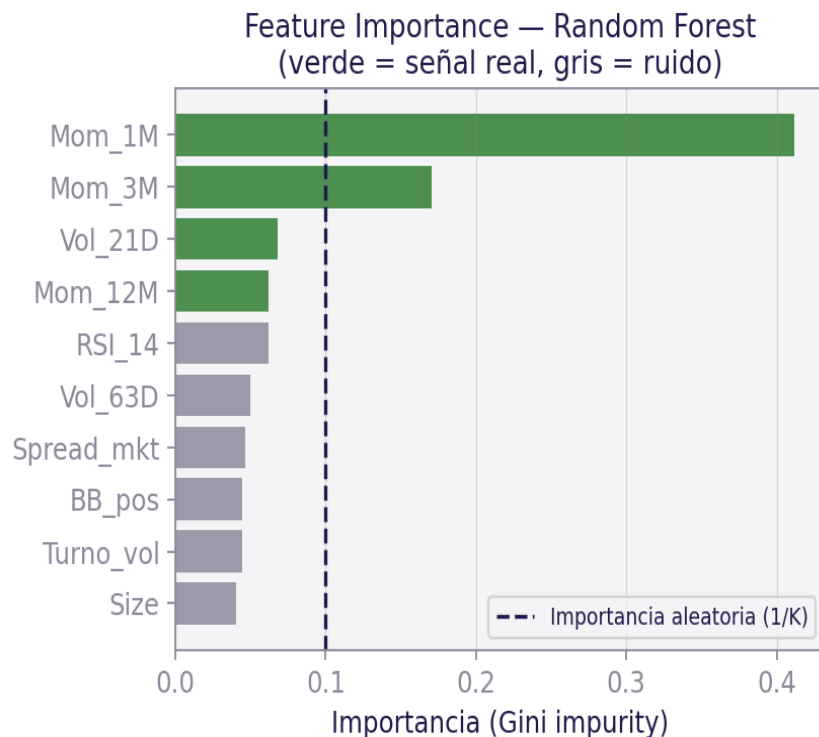
Curva de aprendizaje: ¿necesito más datos o mejor modelo?



Los tres casos posibles graficados con AUC-ROC vs tamaño del training set. Izquierda: underfitting — train y test convergen abajo (el modelo no capta la señal; necesitás más complejidad). Centro: bien calibrado — convergen arriba (más datos no ayudaría mucho). Derecha: overfitting — gran gap que no cierra (el modelo memorizó el train; regularizá más). En finanzas casi siempre vemos el caso de la derecha.

El flujo de trabajo: cada etapa resuelve un problema estructural

gap=37.0pp



El pipeline completo: datos muestreados por actividad (§2) → etiquetas de triple barrera (§3) → pesos por unicidad y diferenciación fraccionaria (§4) → modelo entre los seis candidatos (§6) → validación purgada (§5) → importancia de features con MDI/MDA (§6) → veredicto con DSR/PBO (§7). El rigor está en las puntas — datos, etiquetas, validación — no en el modelo del medio.

- ▶ 1. El pipeline estándar de DS aplicado a finanzas miente: cada supuesto que viola es plata que perdés
- ▶ 2. Muestrear por actividad (dollar bars), no por reloj: los datos son la base de todo lo demás
- ▶ 3. Etiquetá como operás: la triple barrera + barreras adaptativas a la vol del momento
- ▶ 4. La validación es lo más importante: leakage temporal + solapamiento de etiquetas inflan cualquier resultado
- ▶ 5. El modelo es lo de menos: un random forest con buenos datos y buenas etiquetas gana a XGBoost con datos sucios
- ▶ 6. AUC sobre accuracy, MDA sobre MDI, DSR sobre Sharpe: siempre la métrica que no miente
- ▶ 7. Si el DSR deflacta tu Sharpe a 0.3, no es mala suerte — es información: el experimento gastó el dataset sin encontrar señal real